National and Kapodistrian University of Athens (NKUA)
MSc in Data Science and Information Technologies (DSIT)
Course: Database Systems *(2025–2026)*

Students: *Konstantinos Kritsotakis & Vassilis Seitaridis*
Graduate Students: *MSc DSIT – Specialization: Big Data & Artificial Intelligence*

Implementation and Evaluation of D-RAG: A Differentiable Retrieval-Augmented Generation Framework for Knowledge Graph Question Answering

---

Abstract

Knowledge Graph Question Answering (KGQA) systems commonly employ Retrieval-Augmented Generation (RAG) pipelines, where a retriever first selects relevant facts from a knowledge graph and a large language model subsequently generates the final answer. Conventional retrieval mechanisms rely on discrete fact selection, preventing gradients from propagating from the generator back to the retriever during training. As a result, both components are typically optimized independently, limiting the effectiveness of end-to-end learning.

This project presents an implementation of **D-RAG (Differentiable Retrieval-Augmented Generation)**, following the framework proposed by Gao et al. (EMNLP 2025). The implemented system combines a graph neural network-based retriever, differentiable Binary Gumbel-Softmax fact sampling with Straight-Through Estimation (STE), neural fact projection, and an autoregressive language model for answer generation. The implementation follows the proposed two-stage training pipeline, consisting of retriever pretraining followed by joint optimization of the retriever and generator.

The repository includes preprocessing scripts, model training pipelines, evaluation utilities, reproducible execution instructions, and documentation for running experiments on the WebQSP benchmark. The implementation aims to reproduce the principal components of the D-RAG framework while providing a modular codebase suitable for further research on Retrieval-Augmented Generation, Knowledge Graph Question Answering, Graph Neural Networks, and Large Language Models.

Keywords: Retrieval-Augmented Generation, Knowledge Graph Question Answering, Differentiable Retrieval, Graph Neural Networks, Large Language Models, Gumbel-Softmax, Straight-Through Estimation.

List of Figures and Tables

1. Introduction

Knowledge Graph Question Answering (KGQA) aims to answer natural language questions by reasoning over structured knowledge graphs. Unlike traditional text-based question answering systems, KGQA operates on graph-structured data consisting of entities connected through semantic relations. This representation enables multi-hop reasoning and provides interpretable evidence for generated answers.

Recent advances in Large Language Models (LLMs) have significantly improved reasoning capabilities in many natural language processing tasks. Nevertheless, LLMs alone cannot reliably answer knowledge-intensive questions because their internal parameters cannot store or update all factual information contained in large knowledge graphs. Consequently, Retrieval-Augmented Generation (RAG) has emerged as a promising paradigm in which external knowledge is retrieved before answer generation.

Although RAG has demonstrated strong performance across numerous applications, existing KGQA pipelines generally separate retrieval from generation. The retriever selects a discrete set of relevant graph facts, while the generator produces the answer using only the selected evidence. Since the retrieval process involves non-differentiable operations, such as discrete sampling or threshold-based selection, gradients cannot propagate from the generator back to the retriever. As a consequence, both components are trained independently, preventing true end-to-end optimization.

To address this limitation, Gao et al. proposed **D-RAG (Differentiable Retrieval-Augmented Generation)**, a framework that replaces discrete retrieval with differentiable probabilistic fact selection. By employing Binary Gumbel-Softmax sampling together with Straight-Through Estimation (STE), D-RAG enables gradients generated by the language model to update the retriever during training. This allows the retrieval and generation modules to be jointly optimized within a single learning framework.

The objective of this project is to implement the principal components of the D-RAG architecture, reproduce its two-stage training procedure, and evaluate the resulting implementation on the WebQSP benchmark. The repository has been designed to provide a reproducible implementation of the proposed framework while maintaining a modular architecture suitable for future extensions and experimentation.

---

2. Background

2.1 Knowledge Graph Question Answering

Knowledge Graph Question Answering is a task in which a system receives a natural language question and retrieves the corresponding answer by reasoning over a structured knowledge graph. A knowledge graph represents information as triples of the form *(head entity, relation, tail entity)*, enabling explicit modeling of semantic relationships between entities.

Unlike conventional document retrieval systems, KGQA requires identifying the relevant reasoning paths connecting multiple entities within the graph. Many questions involve multi-hop reasoning, where the correct answer can only be obtained after traversing several connected facts. Consequently, the quality of the retrieved subgraph directly influences the accuracy of the generated answer.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines external knowledge retrieval with neural language generation. Given an input question, a retriever first selects relevant evidence from a knowledge source, after which a language model generates the final answer conditioned on the retrieved information.

In knowledge graph applications, the retrieved evidence typically consists of a subgraph containing entities and relations considered relevant to the input question. Although this approach substantially improves factual grounding compared to using an LLM alone, conventional retrieval mechanisms rely on discrete selection operations. Such operations interrupt gradient propagation between the generator and the retriever, forcing both modules to be optimized separately.

2.3 D-RAG Overview

D-RAG addresses the above limitation by introducing differentiable retrieval into the KGQA pipeline. Rather than making hard decisions about whether individual graph facts should be included in the retrieved subgraph, the retriever predicts continuous selection probabilities for each candidate fact. Binary Gumbel-Softmax sampling together with Straight-Through Estimation enables these probabilistic selections to approximate discrete decisions while remaining differentiable during backpropagation.

The retrieved graph representations are subsequently combined with semantic textual representations through a neural projection mechanism before being provided to the language model. This architecture enables gradients produced by the answer generation loss to update both the generator and the graph retriever, allowing joint end-to-end optimization of the complete retrieval-generation pipeline.

---

3. Methodology

3.1 Overall D-RAG Architecture

The implemented system follows the two-stage architecture proposed in the original D-RAG framework. Given a natural language question, the retrieval module first identifies candidate facts from a knowledge graph, while the generation module subsequently produces the final answer conditioned on the retrieved evidence. Unlike conventional Retrieval-Augmented Generation systems, D-RAG enables gradients generated by the language model to propagate through the retrieval process, allowing both modules to be jointly optimized.

The complete pipeline consists of four major components:

- a Graph Neural Network (GNN) retriever,
- a differentiable fact sampler,
- a neural fact projection module,
- an autoregressive language model.

The interaction among these components enables end-to-end optimization of both retrieval and answer generation.

3.2 Graph Retriever

The retrieval module is responsible for estimating the relevance of each candidate fact contained within the retrieved knowledge graph subgraph. Each fact is represented as a knowledge graph triple consisting of a head entity, a relation, and a tail entity.

The retriever employs a Graph Neural Network to encode structural information contained in the graph. By propagating information through neighboring nodes and relations, the model produces contextual embeddings for every candidate fact. A scoring function is then applied to estimate the probability that each fact should contribute to the final answer.

Rather than directly selecting a fixed subset of facts using hard thresholding, the retriever outputs continuous selection probabilities that are later processed by the differentiable sampling module.

3.3 Differentiable Fact Sampling

A major limitation of conventional retrieval systems is the use of discrete selection operations. Binary decisions such as selecting or discarding individual facts interrupt gradient propagation during backpropagation, preventing the language model from improving the retriever.

D-RAG addresses this limitation through Binary Gumbel-Softmax sampling. Instead of performing hard selections during training, each candidate fact is associated with a differentiable sampling probability.

Straight-Through Estimation (STE) is employed to bridge the discrepancy between forward inference and backward optimization. During the forward pass, sampled facts behave similarly to discrete selections, while during backpropagation continuous gradients are propagated through the sampling process.

Consequently, the retrieval module becomes trainable together with the language model.

3.4 Neural Fact Projection

Large language models operate on continuous textual embeddings rather than graph structures. Therefore, graph representations produced by the retriever cannot be directly consumed by the generator.

To address this incompatibility, D-RAG projects graph representations into the language model embedding space. Each retrieved fact is represented through two complementary components:

- semantic information, obtained from the textual representation of the knowledge graph triple,
- structural information, obtained from the Graph Neural Network embeddings.

These representations are fused through a learnable projection layer before being incorporated into the language model input. This allows both semantic content and graph topology to contribute to answer generation.

3.5 Answer Generator

The generator is responsible for producing the final natural language answer conditioned on both the input question and the projected graph representations.

Unlike traditional KGQA pipelines where retrieved facts are simply concatenated as text, D-RAG integrates neural graph representations directly into the language model input through learned embeddings. This enables the language model to exploit both semantic and structural information during autoregressive generation.

During training, the generator is optimized using the standard language modeling objective based on teacher forcing.

3.6 Two-Stage Training Strategy

Following the original D-RAG framework, training is performed in two distinct stages.

Phase 1: Retriever Pretraining

The retrieval module is initially trained independently using heuristic supervision generated from the training dataset. The objective of this stage is to learn meaningful retrieval probabilities before introducing the language model into the optimization process.

The resulting retriever checkpoint provides a stable initialization for the second training stage.

Phase 2: Joint Optimization

After retriever pretraining, the generator and retriever are jointly optimized within a unified training framework.

During this stage, gradients generated by the language modeling loss propagate through the differentiable sampler into the retrieval network. Consequently, both retrieval quality and answer generation are optimized simultaneously.

This joint optimization strategy constitutes the principal contribution of the original D-RAG framework and differentiates it from conventional Retrieval-Augmented Generation systems.

3.7 Loss Functions

Training combines retrieval supervision with language generation objectives.

During Phase 1, optimization focuses exclusively on the retrieval objective using heuristic labels.

During Phase 2, retrieval and generation are jointly optimized. The language modeling objective supervises answer generation, while differentiable retrieval enables gradients to update the retriever through the Binary Gumbel-Softmax sampling mechanism.

This combination allows the complete retrieval-generation pipeline to be optimized end-to-end.

---

4. Implementation

4.1 Repository Organization

The implementation was developed as a modular Python codebase organized around the main components of the D-RAG framework. The repository separates model definitions, training logic, evaluation scripts, preprocessing utilities, documentation, and configuration files. This structure makes the project easier to inspect, reproduce, and extend.

The main implementation directories are summarized in Table 1.

Table 1. Repository structure.

Path	Purpose
`src/model/`	Core model components, including the retriever, differentiable sampler, projector, and generator.
`src/trainer/`	Training and evaluation scripts for Phase 1 retriever training, Phase 2 joint training, and final evaluation.
`scripts/`	Dataset preprocessing and heuristic generation utilities.
`configs/`	Configuration files for related pipelines and experiments.
`data/`	Processed datasets and heuristic supervision files used by the training and evaluation pipeline.
`docs/`	Technical documentation, supplementary notes, and project-related material.
`README.md`	Main documentation file with installation, training, evaluation, and citation instructions.

Path	Purpose
`requirements.txt`	Python dependency specification for reproducible installation.

This organization follows a research-oriented implementation style: individual components can be tested independently, while the complete pipeline remains executable through command-line scripts.

4.2 Implemented Components

The implementation contains the main components required for reproducing the D-RAG training workflow.

Table 2. Implemented D-RAG components.

Component	Implementation role
Graph retriever	Encodes candidate graph facts and predicts fact-level relevance probabilities.
Differentiable sampler	Converts fact probabilities into differentiable selections using Binary Gumbel-Softmax and Straight-Through Estimation.
Projector	Maps graph-derived fact embeddings into the language model embedding space.
Generator	Produces the final answer conditioned on the question and selected graph evidence.
Phase 1 trainer	Performs retriever pretraining using heuristic supervision.
Phase 2 trainer	Performs joint retriever-generator optimization.
Evaluation script	Computes retrieval and generation metrics on the evaluation split.

The implementation therefore focuses on reproducing the principal D-RAG pipeline rather than proposing a new architecture.

4.3 Phase 1 Retriever Training

Phase 1 trains the graph retriever independently before introducing the generator into the optimization loop. This step provides a stable retrieval initialization and follows the two-stage training strategy proposed by the original D-RAG framework.

The retriever is trained using heuristic supervision derived from the processed WebQSP data. Each training instance contains a question, candidate graph facts, and labels indicating which facts are considered relevant. The model learns to assign higher probabilities to relevant facts and lower probabilities to irrelevant facts.

The Phase 1 training command used in this implementation was:

```
python -m src.trainer.train_phase1 \
    --heuristics_path data/train_heuristics_webqsp_subgraph.jsonl \
    --epochs 10 \
    --batch_size 16 \
    --checkpoint_dir checkpoints_webqsp_subgraph_fixed
```

The resulting checkpoint was used as the initialization point for Phase 2 joint training.

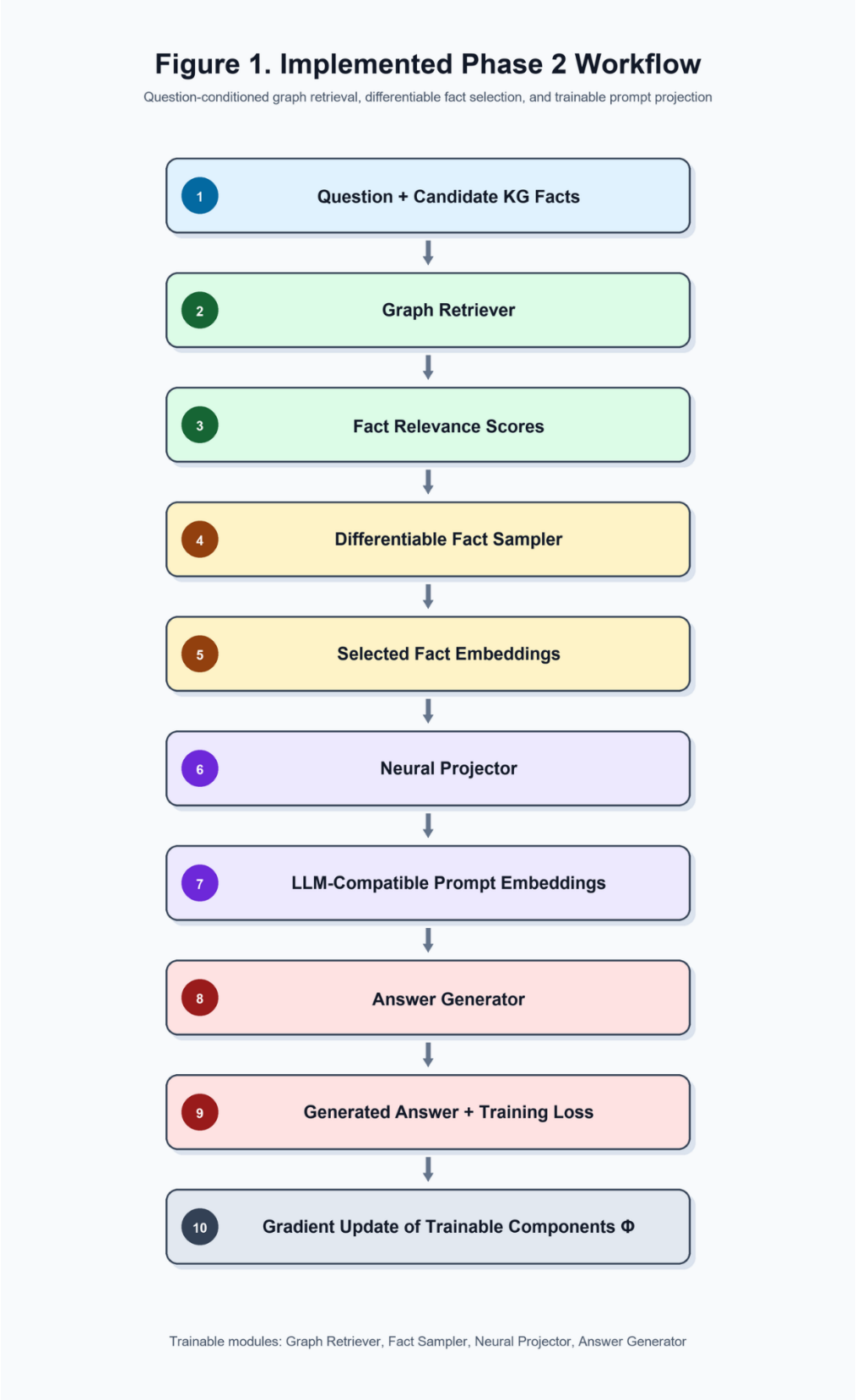
4.4 Phase 2 Joint Training

Phase 2 integrates the pretrained retriever, differentiable sampler, projector, and generator into a joint training pipeline. During this phase, the system no longer treats retrieval and generation as completely independent processes. Instead, selected graph representations influence the language model input, and the resulting generation loss contributes to the optimization of the complete architecture.

The Phase 2 training command used in this implementation was:

```
python -m src.trainer.train_phase2 \
    --heuristics_path data/train_heuristics_webqsp_fixed_train.jsonl
\
    --val_heuristics_path
data/train_heuristics_webqsp_fixed_dev.jsonl \
    --phase1_checkpoint
checkpoints_webqsp_subgraph_fixed/phase1_best.pt \
    --checkpoint_dir checkpoints_webqsp_phase2_clean_qwen \
    --batch_size 1 \
    --epochs 5 \
    --prob_threshold 0.5 \
    --val_generation
```

The complete Phase 2 workflow is shown in Figure 1.



This workflow is the central technical contribution reproduced by the implementation: retrieval is no longer treated as a purely fixed preprocessing step, but as a trainable component integrated into the answer generation pipeline.

4.5 Evaluation Pipeline

The evaluation pipeline was implemented separately from training to allow reproducible assessment of trained checkpoints. The script loads the trained components, applies the same retrieval-selection-projection procedure, and then evaluates generated answers against the reference answers.

The final test evaluation was performed with the standalone Phase 2 evaluation pipeline using the WebQSP test heuristic file and a generation limit of 1628 examples.

A representative evaluation command was:

```
python -m src.trainer.evaluate_phase2 \
    --heuristics_path data/train_heuristics_webqsp_fixed_train.jsonl
\
    --val_heuristics_path data/test_heuristics_webqsp_subgraph.jsonl
\
    --phase1_checkpoint
checkpoints_webqsp_subgraph_fixed/phase1_best.pt \
    --batch_size 1 \
    --prob_threshold 0.5 \
    --val_generation \
    --val_generation_limit 1628
```

The evaluation script reports both retrieval and generation metrics, including Hits@1, exact match, generation F1, retrieval precision, retrieval recall, retrieval F1, loss values, and selected fact counts.

---

5. Experimental Setup

5.1 Dataset

Experiments were performed on the WebQSP benchmark, a standard dataset for Knowledge Graph Question Answering. WebQSP contains natural language questions grounded in Freebase-style knowledge graph facts and is commonly used to evaluate KGQA systems.

The processed training and evaluation files used in this implementation include:

Table 3. Dataset files used in the implementation.

File	Role
`data/train_heuristics_webqsp_subgraph.jsonl`	Phase 1 retriever training supervision.
`data/train_heuristics_webqsp_fixed_train.jsonl`	Phase 2 training split.

File	Role
`data/train_heuristics_webqsp_fixed_dev.jsonl`	Phase 2 validation split.
`data/test_heuristics_webqsp_subgraph.jsonl`	Final test/evaluation split.

Each example contains a question, candidate graph facts, heuristic retrieval labels, and the reference answer.

5.2 Model Configuration

The implementation uses a graph-based retriever, a differentiable fact sampler, a neural projector, and a language model generator. The exact implementation differs from the original paper in some practical details, especially due to available hardware and software constraints, but it preserves the main methodological structure of D-RAG.

Table 4. Main experimental configuration.

Setting	Value
Dataset	WebQSP
Phase 1 epochs	10
Phase 1 batch size	16
Phase 2 epochs	5
Phase 2 batch size	1
Probability threshold	0.5
Evaluation generation limit	1628
Training framework	PyTorch
Execution device	CUDA-enabled GPU
Evaluation split	WebQSP test heuristics

5.3 Evaluation Metrics

The evaluation procedure reports two groups of metrics.

First, answer generation quality is assessed using:

- **Hits@1**, measuring whether a correct answer appears in the generated output,

- **Exact Match (EM)**, measuring strict answer equality,
- **Generation F1**, measuring token-level answer overlap.

Second, retrieval quality is evaluated using:

- **retrieval precision**,
- **retrieval recall**,
- **retrieval F1**,
- **retrieved ratio**,
- **average number of selected facts**.

This separation is important because a KGQA system may retrieve relevant facts without producing the exact final answer, or conversely may generate a partially correct answer despite imperfect retrieval.

---

6. Results

6.1 Final Evaluation Results

The final model was evaluated on the WebQSP test split using the Phase 2 evaluation pipeline.

Table 5. Final WebQSP evaluation results.

Metric	Value
Hits@1	0.262
Exact Match	0.000
Generation F1	0.058
Hits@1 on retrieved subset	0.277
Generation F1 on retrieved subset	0.062
Retrieved ratio	0.187
Retrieval precision	0.187
Retrieval recall	0.052
Retrieval F1	0.080
Average selected facts	1.0
Minimum selected facts	1

Metric	Value
Maximum selected facts	1
Evaluated generation examples	1581

These results should be interpreted as implementation-level validation rather than a full reproduction of the paper-level benchmark performance. The main goal of this project was to reproduce the complete D-RAG training and evaluation pipeline under the available computational constraints.

6.2 Interpretation of Results

The final evaluation confirms that the implemented system can execute the full retrieval-generation workflow end-to-end. The model successfully loads the trained checkpoints, applies graph-based retrieval, selects facts, projects graph representations into the generator space, produces answers, and computes both retrieval and generation metrics.

The retrieval metrics indicate that the selected facts are sparse. The average number of selected facts is one, which reflects the conservative thresholding strategy used in the final evaluation. This behaviour reduces retrieval noise but also limits recall. As a result, retrieval precision is higher than retrieval recall, while the retrieval F1 remains low.

Generation metrics also remain below the levels reported in the original D-RAG paper. This is expected because the implementation was constrained by available computational resources, shorter training, smaller practical batch sizes, and implementation-level differences from the original experimental setup.

The zero exact-match score should be interpreted carefully. The generated outputs often contain verbose textual continuations rather than strictly normalized short answers. Therefore, exact-match evaluation is especially strict in this setting, while Hits@1 and token-level F1 provide additional information about answer overlap.

The most important outcome is therefore not numerical parity with the paper, but successful reproduction of the main training and evaluation mechanism.

6.3 Comparison with the Original Paper

The original D-RAG paper reports substantially higher benchmark results on WebQSP and CWQ. However, those results correspond to the authors' full experimental configuration, model choices, training schedule, and implementation details.

This project should therefore be understood as an educational and research-oriented reproduction of the method rather than an exact reproduction of the original benchmark table.

Table 6. Scope comparison between the original paper and this implementation.

Aspect	Original D-RAG paper	This implementation
Main objective	Propose a new differentiable RAG framework for KGQA	Reproduce the main components of the framework
Datasets	WebQSP and CWQ	WebQSP
Training strategy	Retriever pretraining + joint end-to-end optimization	Retriever pretraining + Phase 2 joint optimization
Differentiable sampling	Gumbel-Softmax-based differentiable subgraph sampling	Binary Gumbel-Softmax-style differentiable fact sampling
Prompting	Semantic + structural neural prompting	Neural fact projection into generator space
Evaluation	Full benchmark comparison against baselines	Implementation-level evaluation on WebQSP
Goal	State-of-the-art KGQA performance	Correct, reproducible implementation of the pipeline

This comparison clarifies that the project reproduces the core methodology, while not claiming to reproduce the exact state-of-the-art benchmark numbers from the original publication.

---

7. Discussion

The implementation demonstrates that the main conceptual idea of D-RAG can be translated into a modular software pipeline. The system combines graph retrieval, differentiable fact selection, neural projection, and language generation into a single trainable architecture.

The central technical challenge was the integration of graph-based representations with a language model. Knowledge graph facts are naturally structured as triples, while language models operate on token embeddings. The projector module serves as the bridge between these two representation spaces.

A second challenge was maintaining differentiability through the retrieval process. Conventional retrieval pipelines rely on hard selection, which blocks gradients. By using differentiable fact sampling, the implemented pipeline allows retrieval decisions to participate in the optimization process.

A third practical challenge was computational efficiency. Jointly training a retriever and a generator is significantly more expensive than training either component

independently. This required using small batch sizes, checkpoint management, generation limits during evaluation, and conservative inference settings.

Overall, the implementation provides a functional and extensible reproduction of the D-RAG pipeline. Its main value lies in demonstrating the complete mechanism of differentiable retrieval-augmented generation for knowledge graph question answering.

---

8. Limitations

This implementation has several limitations.

First, the reported results do not match the performance reported in the original D-RAG paper. This is expected because the goal of the project was implementation and pipeline reproduction, not full-scale benchmark reproduction.

Second, the experiments were limited to WebQSP. The original paper also evaluates on CWQ, which contains more complex multi-hop questions.

Third, the final evaluation used a conservative probability threshold, leading to sparse fact selection. While this reduces retrieval noise, it also limits recall and may reduce the amount of useful evidence available to the generator.

Fourth, computational constraints influenced several implementation choices, including batch size, number of training epochs, and validation generation limits.

Fifth, exact-match performance remained zero in the final evaluation. This suggests that additional answer normalization, constrained decoding, or post-processing would be needed for more reliable strict-match evaluation.

Finally, the current report does not claim that the implementation outperforms existing KGQA systems. It claims that the principal D-RAG architecture and training procedure were implemented and evaluated.

---

9. Future Work

Future work could improve the implementation in several directions:

1. Evaluate the pipeline on CWQ in addition to WebQSP.
2. Tune the probability threshold to better balance retrieval precision and recall.
3. Compare multiple generator backbones.
4. Add systematic ablation experiments for the sampler, projector, and retriever.
5. Improve answer post-processing for exact match and F1 evaluation.
6. Explore larger-scale training with longer Phase 2 optimization.
7. Add visualization tools for selected facts and retrieval probabilities.

8. Extend the framework to biomedical knowledge graphs, where drug-protein-disease reasoning requires structured multi-hop retrieval.

The final direction is particularly promising because biomedical knowledge graphs naturally contain structured relational facts, such as drug-target, protein-pathway, and disease-association relations. A differentiable retrieval mechanism could help reduce irrelevant graph evidence while preserving biologically meaningful reasoning paths.

---

10. Conclusion

This project implemented and evaluated D-RAG, a differentiable retrieval-augmented generation framework for Knowledge Graph Question Answering. The implementation includes graph-based retrieval, differentiable fact sampling, neural fact projection, Phase 1 retriever training, Phase 2 joint optimization, and final evaluation on WebQSP.

The final system does not claim to reproduce the exact benchmark performance of the original publication. Instead, it provides a faithful implementation of the principal D-RAG pipeline and demonstrates how differentiable retrieval can be integrated with language model generation.

The resulting repository is modular, documented, reproducible, and suitable for future research extensions in Retrieval-Augmented Generation, Knowledge Graph Question Answering, Graph Neural Networks, and Large Language Models.

References

[1] Gao, G., Li, Z., Yuan, C., Li, J., Wu, J., Zhang, Y., Jin, X., Li, B., and Hu, W. (2025). *D-RAG: Differentiable Retrieval-Augmented Generation for Knowledge Graph Question Answering*. Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), 35398–35417. Association for Computational Linguistics. DOI: 10.18653/v1/2025.emnlp-main.1793.

[2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS).

[3] Jang, E., Gu, S., and Poole, B. (2017). *Categorical Reparameterization with Gumbel-Softmax*. International Conference on Learning Representations (ICLR).

[4] Maddison, C. J., Mnih, A., and Teh, Y. W. (2017). *The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables*. International Conference on Learning Representations (ICLR).

[5] Kipf, T. N., and Welling, M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. International Conference on Learning Representations (ICLR).

[6] Yih, W., Richardson, M., Meek, C., Chang, M., and Suh, J. (2016). *The Value of Semantic Parse Labeling for Knowledge Base Question Answering*. Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL).

Appendix A. Reproducibility Commands

Phase 1 Retriever Training

```
python -m src.trainer.train_phase1 \
    --heuristics_path data/train_heuristics_webqsp_subgraph.jsonl \
    --epochs 10 \
    --batch_size 16 \
    --checkpoint_dir checkpoints_webqsp_subgraph_fixed
```

Phase 2 Joint Training

```
python -m src.trainer.train_phase2 \
    --heuristics_path data/train_heuristics_webqsp_fixed_train.jsonl
\
    --val_heuristics_path
data/train_heuristics_webqsp_fixed_dev.jsonl \
    --phase1_checkpoint
checkpoints_webqsp_subgraph_fixed/phase1_best.pt \
    --checkpoint_dir checkpoints_webqsp_phase2_clean_qwen \
    --batch_size 1 \
    --epochs 5 \
    --prob_threshold 0.5 \
    --val_generation
```

Final Evaluation

```
python -m src.trainer.evaluate_phase2 \
    --heuristics_path data/train_heuristics_webqsp_fixed_train.jsonl
\
    --val_heuristics_path data/test_heuristics_webqsp_subgraph.jsonl
\
    --phase1_checkpoint
checkpoints_webqsp_phase2_clean_qwen/phase2_best.pt \
    --generator_checkpoint
checkpoints_webqsp_phase2_clean_qwen/generator_best \
    --batch_size 1 \
    --prob_threshold 0.5 \
    --val_generation \
    --val_generation_limit 1628
```